

THEORY OF ARTIFICIAL INTELLIGENCE

Lecture 5 — From Logic to Probability

Classification Task: Logistic Regression and Perceptron

When Rules Don't Have One Outcome

- Toothache $\Rightarrow$ Cavity — wrong: gum disease, an abscess, and other causes exist too
- Cavity $\Rightarrow$ Toothache — also wrong: not every cavity causes pain
- **Fixing this exhaustively runs into three walls:**
 - Laziness — listing every antecedent/consequent is too much work
 - Theoretical ignorance — medicine has no complete causal theory here
 - Practical ignorance — not every test has been run on this patient

The agent's knowledge can, at best, provide a degree of belief — our tool for that is probability.

From the Joint to a Simple 2×2 Table

The full joint over Toothache, Cavity is a 2×2 table

	toothache	¬toothache
cavity	0.12	0.08
¬cavity	0.08	0.72

Two rules we'll use next

Chain rule: $P(\text{cavity} \wedge \text{toothache}) = P(\text{cavity} \mid \text{toothache}) \cdot P(\text{toothache}) = P(\text{toothache} \mid \text{cavity}) \cdot P(\text{cavity})$

Bayes' rule: $P(\text{cavity} \mid \text{toothache}) = P(\text{toothache} \mid \text{cavity}) \cdot P(\text{cavity}) / P(\text{toothache})$

Computing $P(\text{Cavity} \mid \text{Toothache})$

- Read $P(\text{cavity} \wedge \text{toothache})$ directly off the table: 0.12
- Marginalize to get $P(\text{toothache})$ — sum the whole toothache column:

$$P(\text{toothache}) = P(\text{cavity} \wedge \text{toothache}) + P(\neg \text{cavity} \wedge \text{toothache}) = 0.12 + 0.08 = 0.20$$

- **Now apply Bayes' rule:**

$$P(\text{cavity} \mid \text{toothache}) = 0.12 / 0.20 = 0.6$$

Given a toothache, there's a 60% chance of a cavity — real, useful, and nowhere near the certainty a logical rule would have demanded.

Decision Theory = Probability + Utility

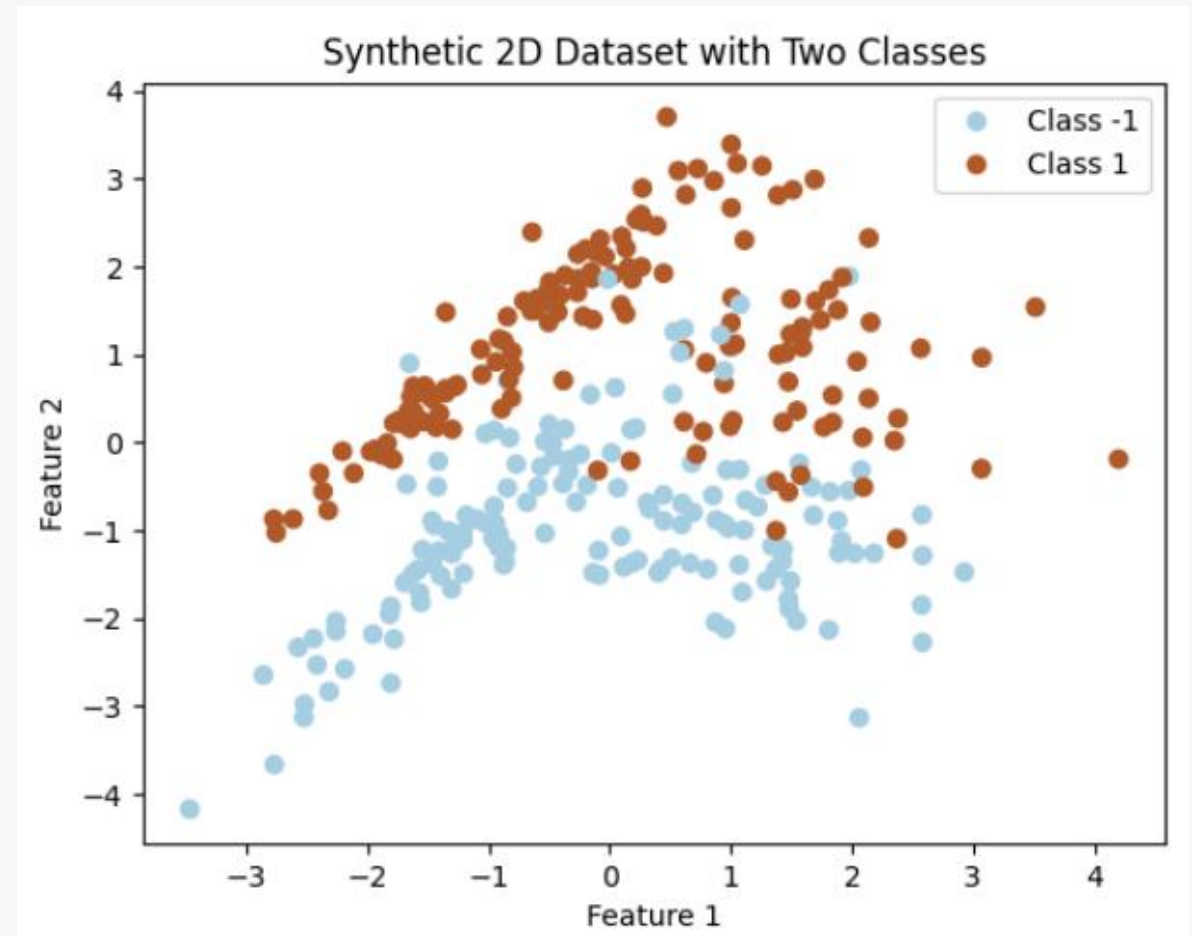
- An agent is rational iff it chooses the action with highest expected utility, averaged over outcomes
- This is the Principle of Maximum Expected Utility (MEU)

```
function DT-AGENT(percept) returns an action  
  persistent: belief_state, probabilistic beliefs about the current state of the world  
               action, the agent's action  
  
  update belief_state based on action and percept  
  calculate outcome probabilities for actions,  
    given action descriptions and current belief_state  
  select action with highest expected utility  
    given probabilities of outcomes and utility information  
  return action
```

Figure 12.1 A decision-theoretic agent that selects rational actions.

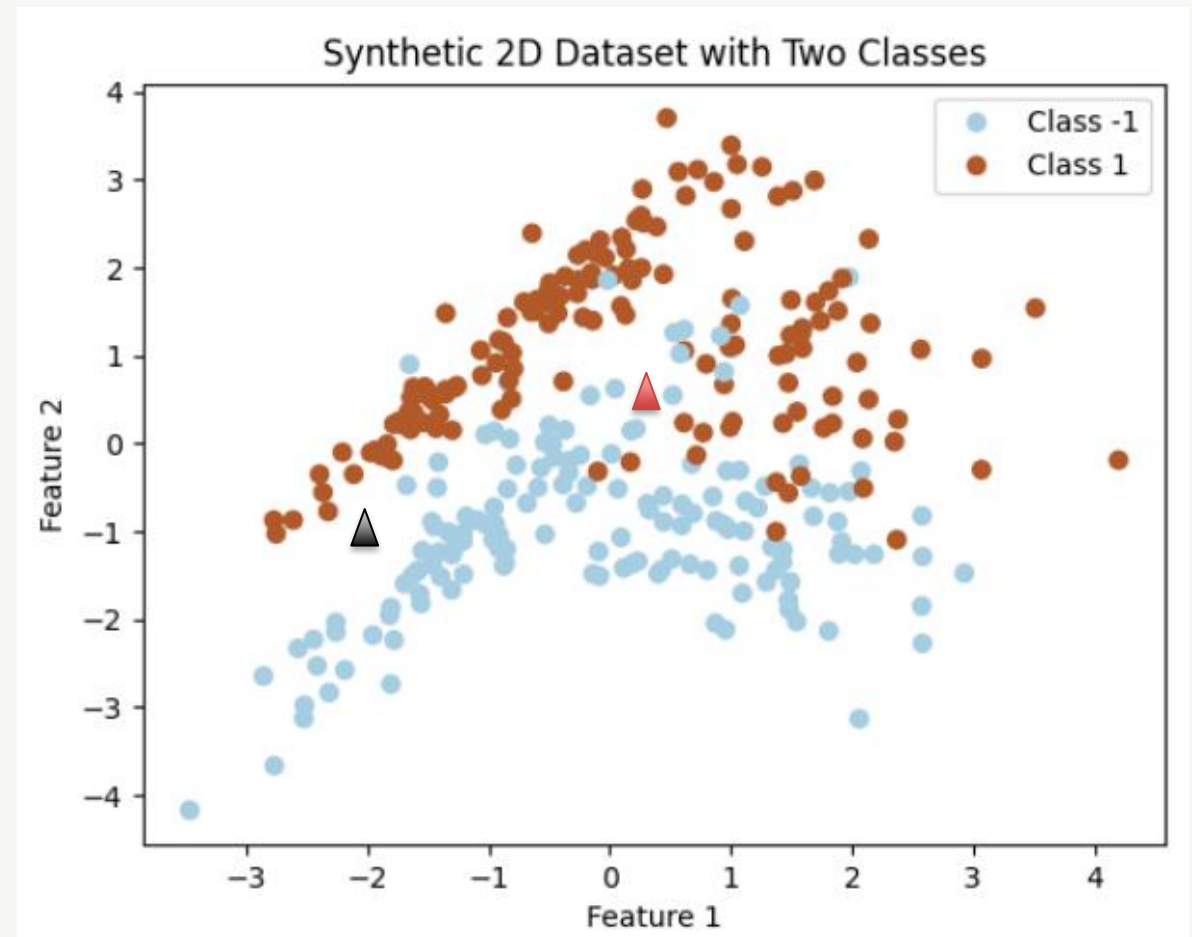
The Classification Task

- We're given points $(x^{(i)}, y^{(i)})$, $i = 1, \dots, N$
- $y^{(i)} \in \{-1, 1\}$ — a binary label per point
- $x^{(i)} \in \mathbb{R}^2$ for our running example — two features

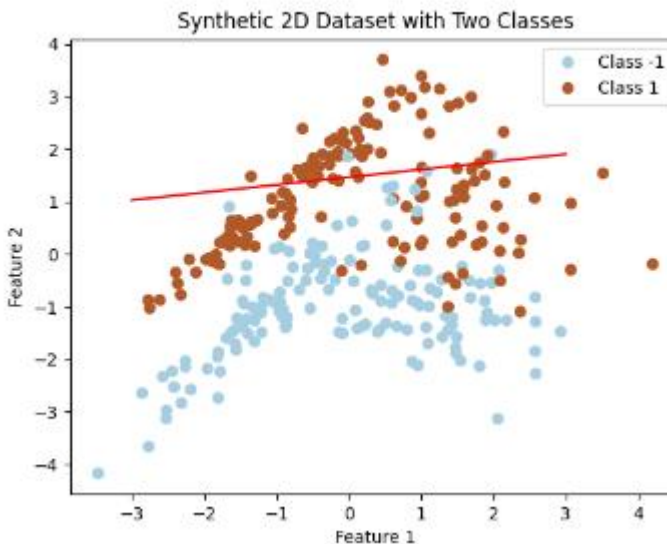
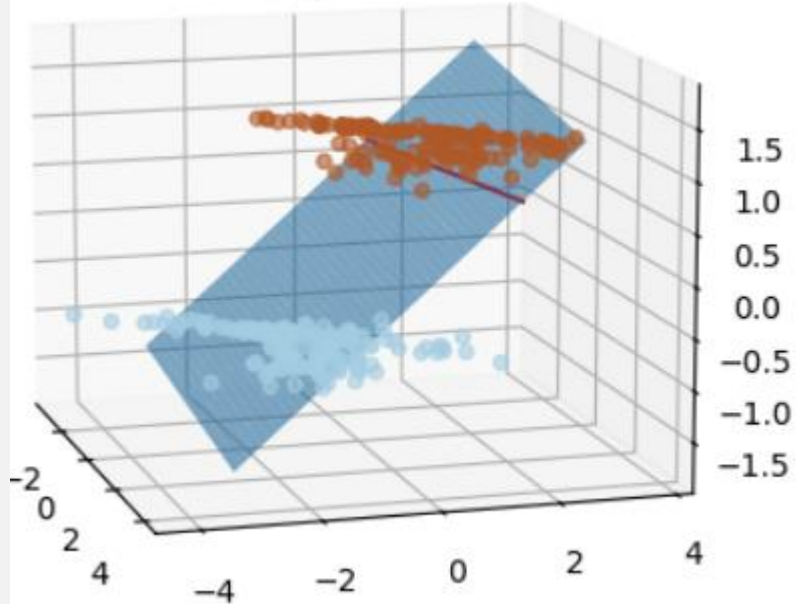
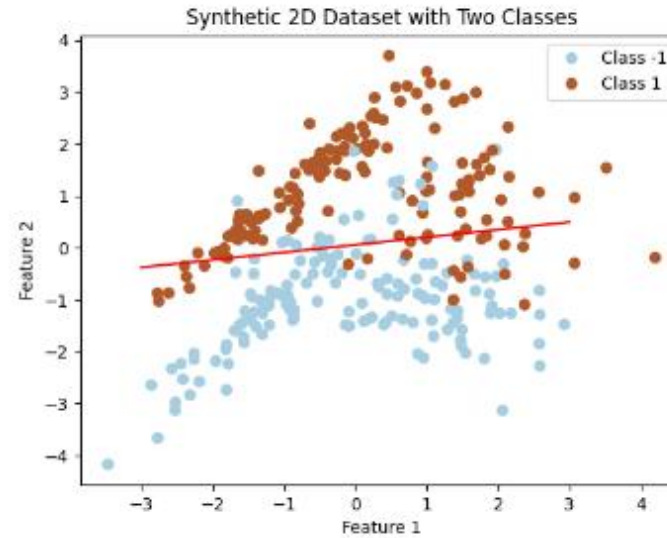
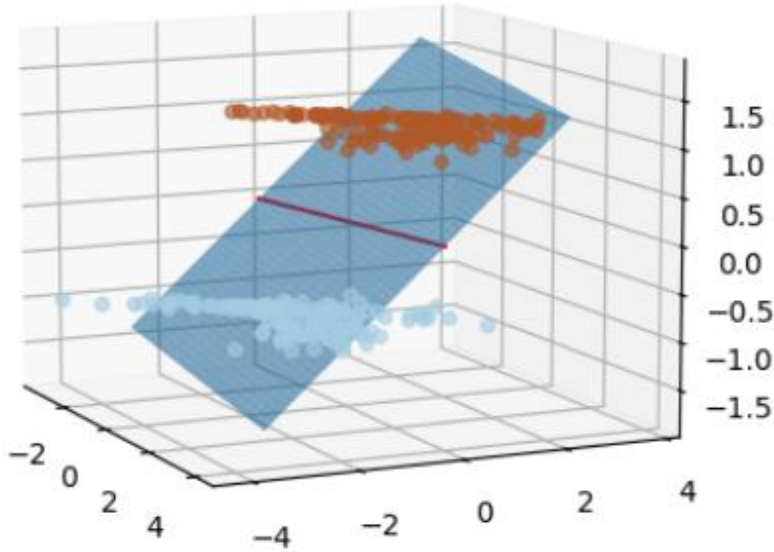


What the Agent Must Decide

- Given a new point, our algorithm — our agent — must say which class it belongs to
- $a(\triangle) = +1$ or -1
- This is a decision rule: a function from features to an action (a label)



A Linear Model for Classification



- A hyperplane $w_0 + w_1x_1 + w_2x_2 = 0$ splits the plane into two half-spaces
- Points where the expression is positive get one label, negative the other

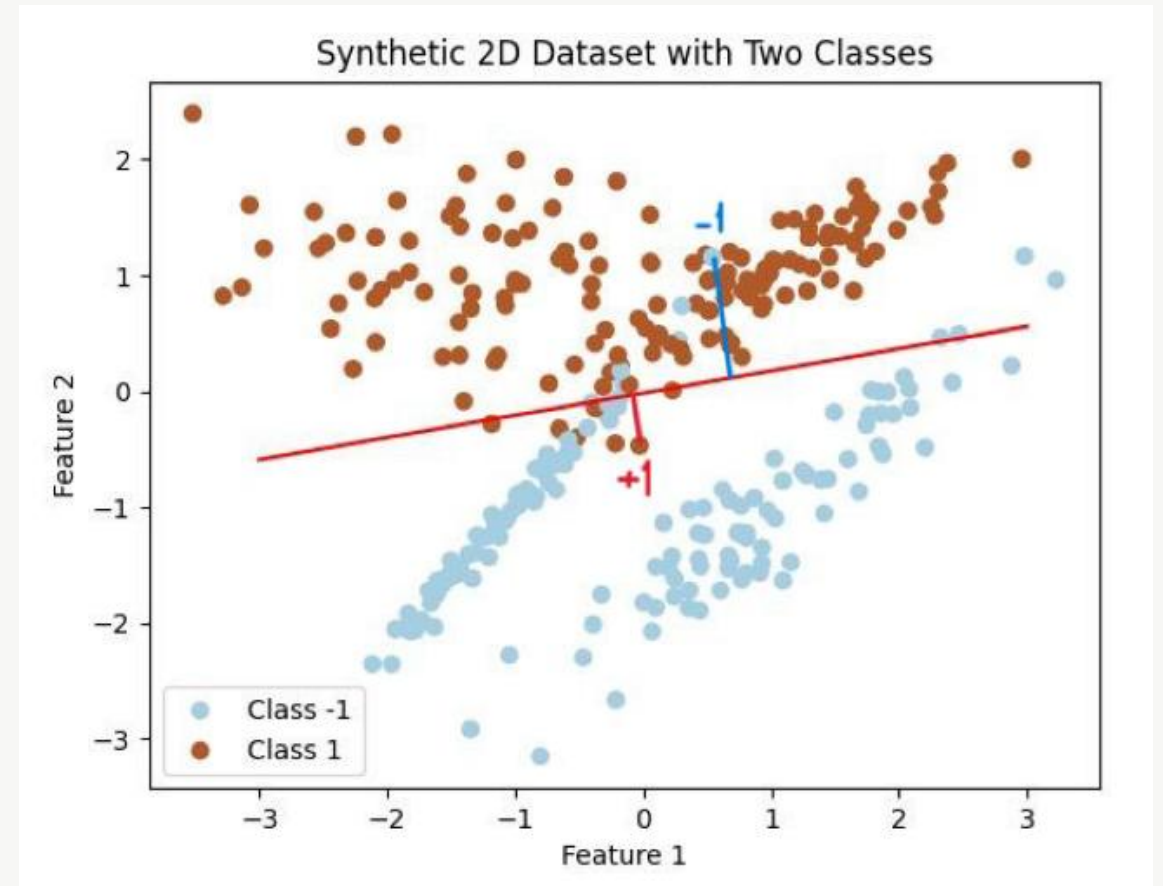
Signed Distance to the Boundary

- The distance from point (x_1, x_2) to the line:

$$d = \frac{w_0 + w_1x_1 + w_2x_2}{\sqrt{w_1^2 + w_2^2}} = \frac{\langle w, \tilde{x} \rangle}{\sqrt{w_1^2 + w_2^2}}$$

- Sign tells you which side;
- magnitude tells you how far

using the augmented vector $\tilde{x} = (1, x_1, x_2)$



The Margin: Distance, Signed by Correctness

- The margin of x is the distance multiplied by the true sign y :

$$\text{margin} = y \cdot \langle w, \tilde{x} \rangle$$

Correct prediction

$$\begin{aligned} y=1, \langle w, \tilde{x} \rangle > 0 &\rightarrow \text{margin} > 0 \\ y=-1, \langle w, \tilde{x} \rangle < 0 &\rightarrow \text{margin} > 0 \end{aligned}$$

Incorrect prediction

$$\begin{aligned} y=1, \langle w, \tilde{x} \rangle < 0 &\rightarrow \text{margin} < 0 \\ y=-1, \langle w, \tilde{x} \rangle > 0 &\rightarrow \text{margin} < 0 \end{aligned}$$

Accuracy Isn't Enough

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$precision = \frac{TP}{TP + FP}$$

$$recall/sensitivity/TPR = \frac{TP}{TP + FN} = \frac{TP}{Total\ positive} = \frac{TP}{P}$$

		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection

- Accuracy = fraction of correct predictions — but it hides class imbalance
- N=100, 95 points labeled +1, 5 labeled -1: predicting everything +1 scores 95% accuracy
- If q is the largest class's fraction, a useful model needs Accuracy $\in [q, 1]$

Precision and Recall

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$$

Precision

“When I predict positive,
how often am I right?”

Medical test: of everyone
flagged positive, what
fraction is truly sick?

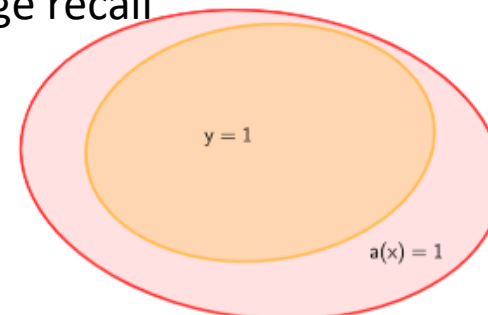
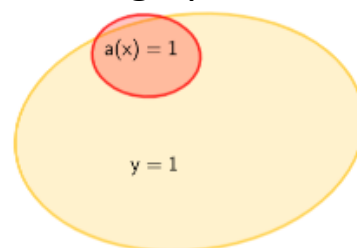
$$\text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

Recall (sensitivity, TPR)

“Of everyone who's actually
positive, how many did I catch?”

Missing this = a sick
patient told they're fine

Large precision VS Large recall



Model Selection

$a(x) = \langle w, x \rangle$	y	$\hat{y}$
1000	1	1
900	-1	1
800	1	1
700	1	-1
300	-1	-1
100	-1	-1
1	-1	-1
-10	1	-1
-200	-1	-1
-500	-1	-1

$a(x) = \langle w, x \rangle$	y	$\hat{y}$
1000	1	1
900	-1	1
800	1	1
700	1	1
300	-1	1
100	-1	1
1	-1	1
-10	1	1
-200	-1	-1
-500	-1	-1

Model Selection

$a(x) = \langle w, x \rangle$	y	$\hat{y}$
1000	1	1
900	-1	1
800	1	1
700	1	-1
300	-1	-1
100	-1	-1
1	-1	-1
-10	1	-1
-200	-1	-1
-500	-1	-1

	Predicted +	Predicted -
Actual +	TP=2	FN=2
Actual -	FP=1	TN=5

precision = $TP / (TP+FP)=2/3$

recall = $TP / (TP+FN)=2/6=1/3$

$a(x) = \langle w, x \rangle$	y	$\hat{y}$
1000	1	1
900	-1	1
800	1	1
700	1	1
300	-1	1
100	-1	1
1	-1	1
-10	1	1
-200	-1	-1
-500	-1	-1

	Predicted +	Predicted -
Actual +	TP=4	FN=0
Actual -	FP=4	TN=2

precision = $TP / (TP+FP)=4/8=1/2$

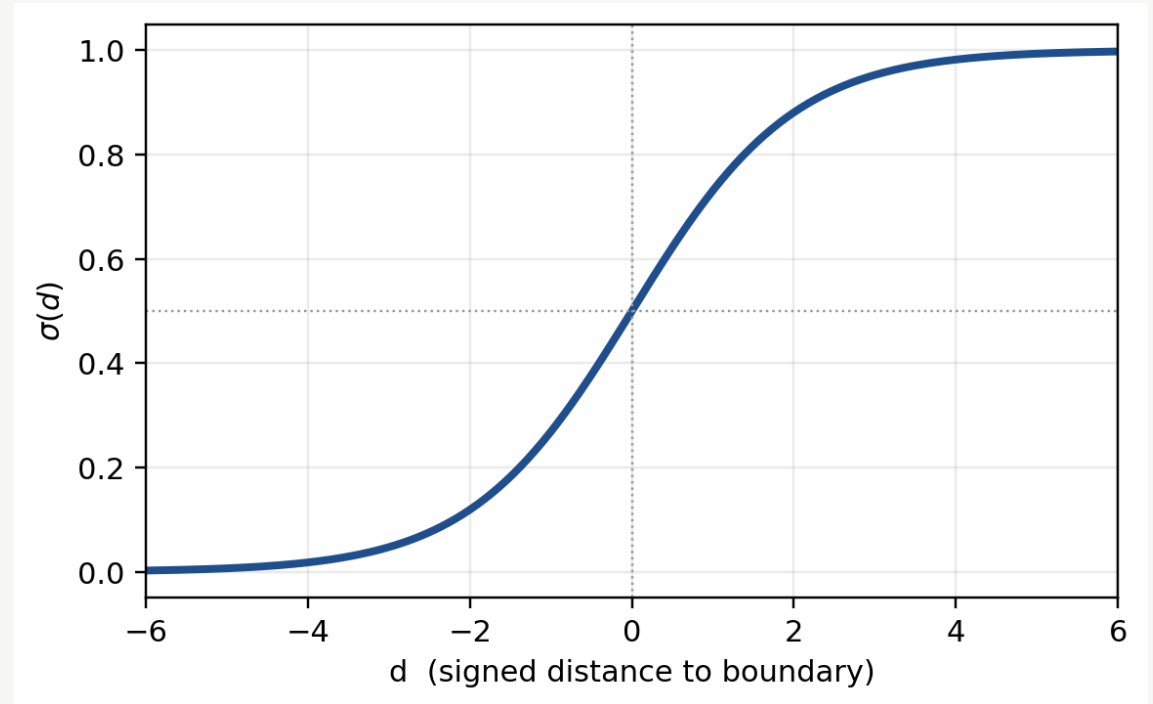
recall = $TP / (TP+FN)=4/6=2/3$

Turning a Distance Into a Probability

- The signed distance d is unbounded — not a probability
- Map it through σ : $(-\infty, +\infty) \rightarrow (0, 1)$:

$$\sigma(d) = 1 / (1 + e^{-d})$$

- $d \gg 0 \rightarrow \sigma(d) \approx 1$
- $d = 0 \rightarrow \sigma(d) = 0.5$
- $d \ll 0 \rightarrow \sigma(d) \approx 0$



Log-Odds: Inverting the Sigmoid

- Solve $\sigma(d)$ for d :

$$d = \log(\sigma / (1-\sigma))$$

- Since σ is the probability of the positive class, this is the log-ratio of the two class probabilities
- **Known as the log-odds, or logit**
- This is the bridge that turns a plain geometric distance into a genuine probabilistic model

A Genuine Probabilistic Model

- For every x , Y is a random variable: $+1$ with probability $p(x)$, -1 with probability $1-p(x)$
- Assume $p(x) = \sigma(w_0 + w_1 x_1 + \dots + w_d x_d)$

The model, compactly

$$P(Y=1 ; x, w) = \sigma(\langle w, \tilde{x} \rangle) = 1/(1+e^{-\langle w, \tilde{x} \rangle})$$

$$P(Y=-1 ; x, w) = 1-\sigma(\langle w, \tilde{x} \rangle) = 1/(1+e^{\langle w, \tilde{x} \rangle})$$

$$L(w) = \prod_{i=1}^N \sigma \left(y^{(i)} \langle w, x^{(i)} \rangle \right) = \prod_{i=1}^N \frac{1}{1 + e^{-y^{(i)} \langle w, x^{(i)} \rangle}} \rightarrow \max_w.$$

Maximum Likelihood → Binary Cross-Entropy Loss

- Maximizing $\log L(w)$ is the same as minimizing $-\log L(w)$:

$$Loss = \frac{1}{N} \sum_{i=1}^N \log \left(1 + e^{-y^{(i)} \langle w, x^{(i)} \rangle} \right)$$

Binary cross-entropy loss

The definition of entropy, cross-entropy, and KL-divergence are as follows

$$H(p) = - \sum_{i=1}^K p_i \log p_i, \quad H(p, q) = - \sum_{i=1}^K p_i \log q_i, \quad D_{KL}(p, q) = \sum_{i=1}^K p_i \log \frac{q_i}{p_i}.$$

It follows that

$$H(p, q) = H(p) + D_{KL}(p, q).$$

With p (the true labels) fixed, minimizing cross-entropy loss is exactly minimizing KL divergence — pulling the model's predicted distribution toward the true one.

F. Rosenblatt's Perceptron (1962)



The Mark I Perceptron — custom-built hardware, not software, learning a linear classifier directly from wiring.

C. Bishop, Pattern Recognition and Machine Learning, Sec. 4.1.7

<https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>

From 0/1 Loss to the Perceptron Loss

- 0/1 loss just counts mistakes — not useful for gradient-based optimization (flat almost everywhere)
- **Perceptron loss: sum the (negative) margin, but only over misclassified points**

$$Loss = \frac{1}{N} \sum_{i=1}^N \left[\text{sign } a(x^{(i)}) \neq y^{(i)} \right] = \frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \langle w, x^{(i)} \rangle < 0 \right]$$

Perceptron:

$$Loss = \sum_{i: y^{(i)} \langle w, x^{(i)} \rangle < 0} y^{(i)} \langle w, x^{(i)} \rangle \rightarrow \max$$

or

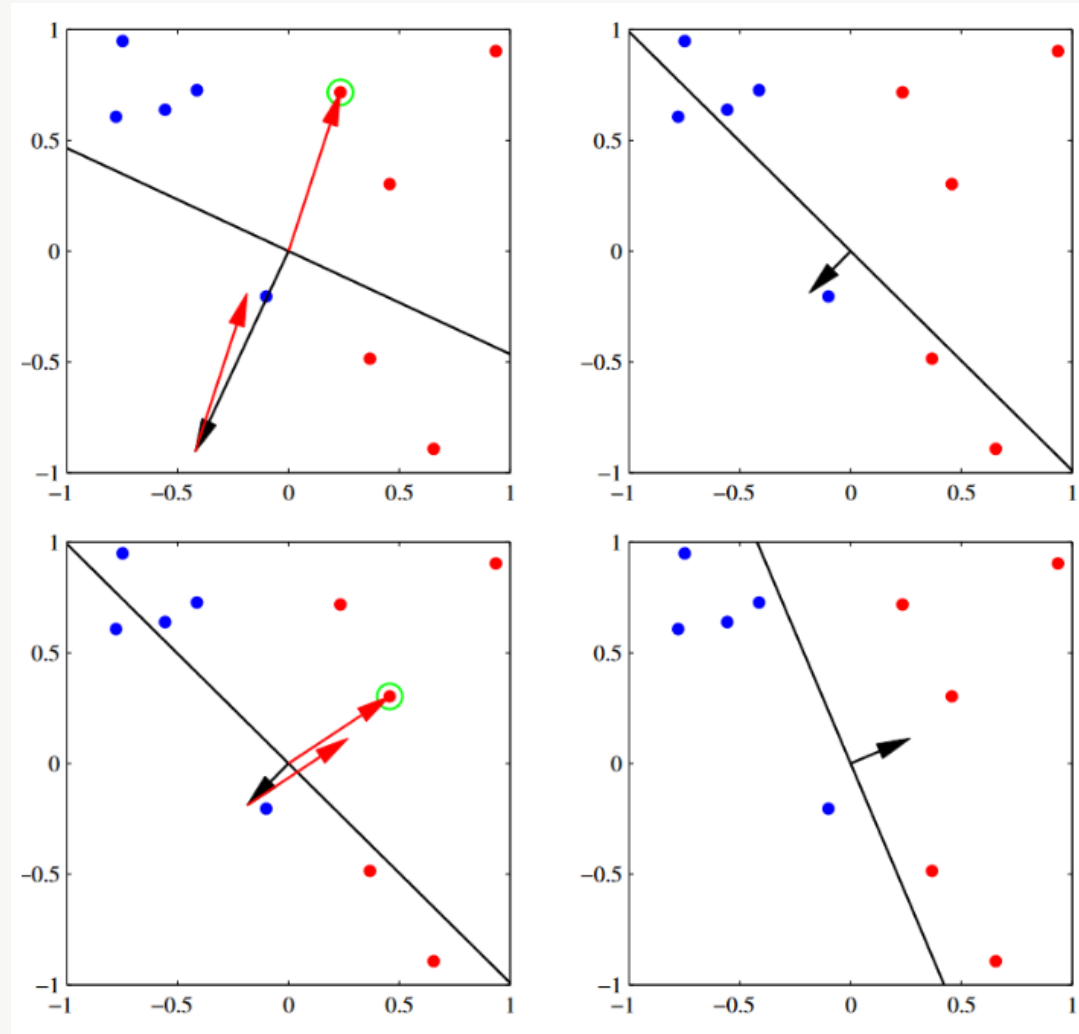
$$Q(w) = - \sum_{i: y^{(i)} \langle w, x^{(i)} \rangle < 0} y^{(i)} \langle w, x^{(i)} \rangle \rightarrow \min .$$

$$Q(w) = - \sum_{i: y^{(i)} \langle w, x^{(i)} \rangle < 0} y^{(i)} \langle w, x^{(i)} \rangle \rightarrow \min .$$

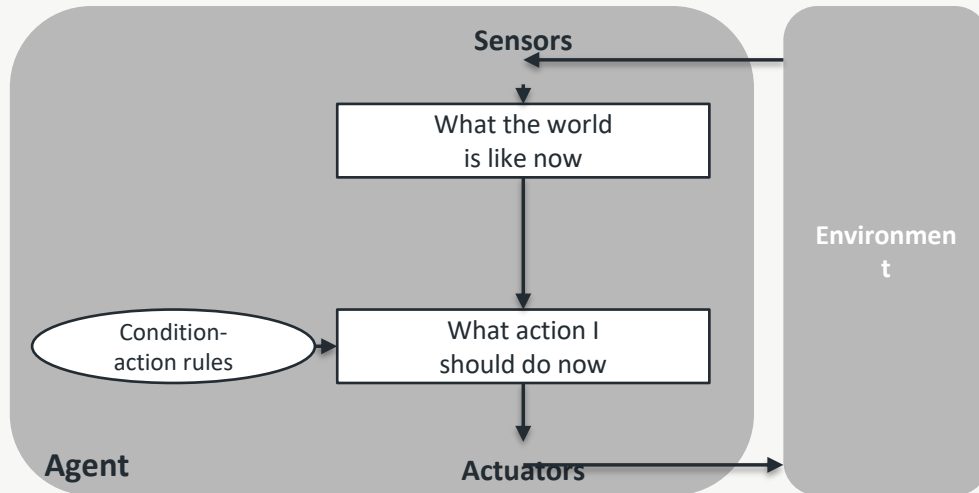
SGD implies the update, if i th point is missclassified:

$$w^{new} = w^{old} + y^{(i)} x^{(i)}$$

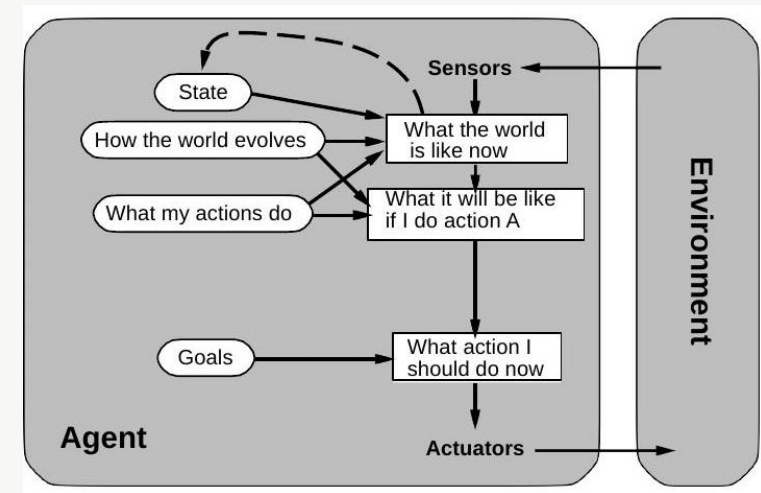
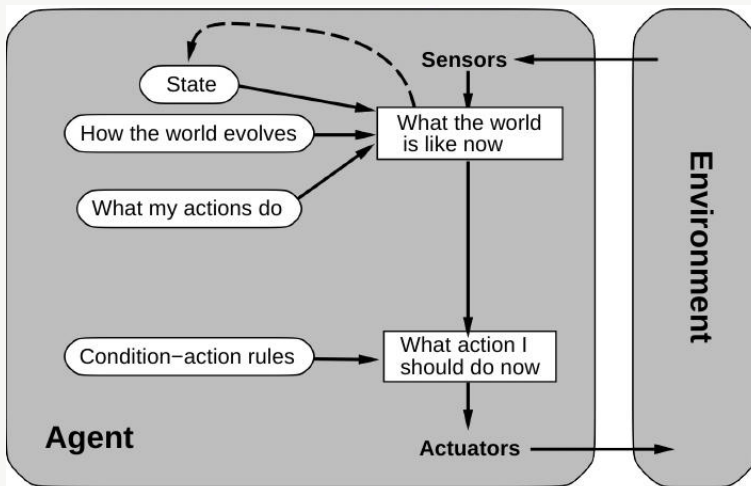
Watching the Boundary Rotate



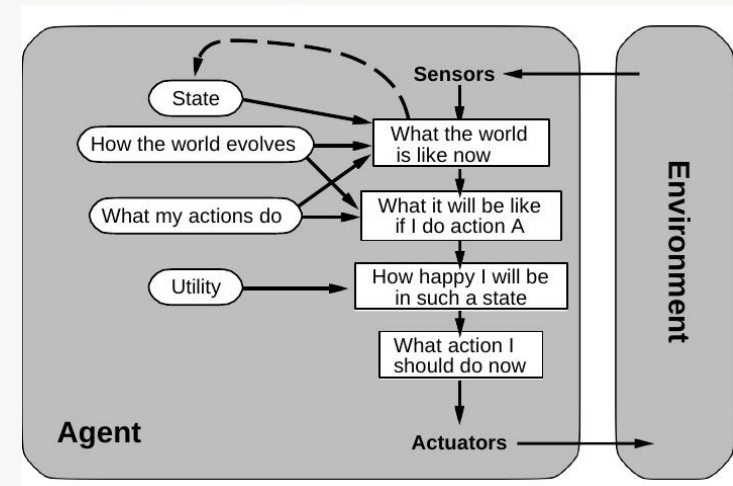
Four Agent Architectures



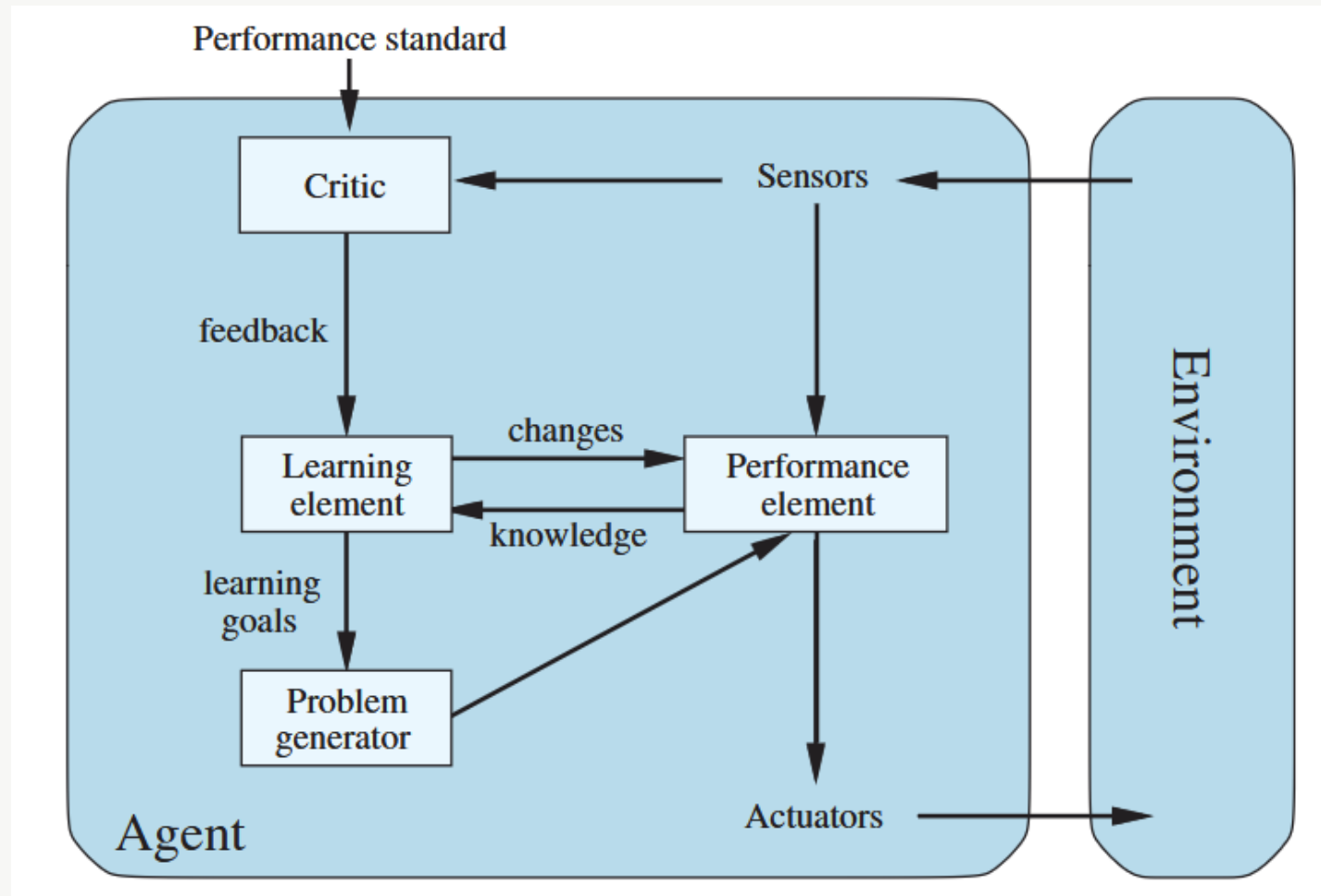
Simple reflex



Goal-based



...as a Learnable Agent



Is the Classifier We Just Built an Agent?

- **Yes — precisely a simple reflex agent: percept (features x) $\rightarrow$ action (label), no internal state**
- PEAS: Performance = accuracy or expected utility; Environment = incoming examples; Actuators = the predicted label; Sensors = feature extraction
- **Add the training loop — gradient descent as the learning element, the loss as the critic — and it becomes a (partial) learning agent**
- Missing only a problem generator — exactly what exploration adds, when we reach bandits